

Introduction to Intensive Programming in C++

Lecture 5: Dynamic programming

Elias TSIGARIDAS

École Polytechnique

What is Dynamic Programming?

Dynamic Programming

An algorithmic technique about smart recursion!

- Optimal substructure.
- Repeated subproblems.

Contents

Fibonacci

WIS

LIS

Edit distance

Knapsack



Calculate the n -th Fibonacci number

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, ...

main technique
in DP

$$F_n = F_{n-1} + F_{n-2}$$

RECFIBO(n):

```
if  $n = 0$ 
  return 0
else if  $n = 1$ 
  return 1
else
  return RECFIBO( $n - 1$ ) + RECFIBO( $n - 2$ )
```

$F_0 = 0$
 $F_1 = 1$ } initial conditions

Compute using the subproblems

Complexity $O(2^n)$

MEMFIBO(n):

```
if  $n = 0$ 
  return 0
else if  $n = 1$ 
  return 1
else
  if  $F[n]$  is undefined
     $F[n] \leftarrow \text{MEMFIBO}(n - 1) + \text{MEMFIBO}(n - 2)$ 
  return  $F[n]$ 
```

memoization
[Remember everything]

$O(n)$

ITERFIBO(n):

```
 $F[0] \leftarrow 0$   
 $F[1] \leftarrow 1$   
for  $i \leftarrow 2$  to  $n$   
   $F[i] \leftarrow F[i - 1] + F[i - 2]$   
return  $F[n]$ 
```

draw the tree

seq of numbers $\rightarrow \begin{cases} \emptyset \\ \text{number, seq of numbers} \end{cases}$

A three-step recipe

- Identify a "small" collection of subproblems.
There should be easy to solve (usually).
- Deduce the solution of the "larger" subproblems from the solutions of the "smaller" ones.
- Deduce the final solution from the solutions of the subproblems.

⚠ Do not forget the initial conditions

- describe the solution
in english/french
- write the recursion
- write the pseudocode

Contents

Fibonacci

WIS

LIS

Edit distance

Knapsack



Weighted Independent Set (WIS) on a path graph

YES $\{v_1, v_3, v_5\}$
NO $\{v_1, v_2, v_5\}$

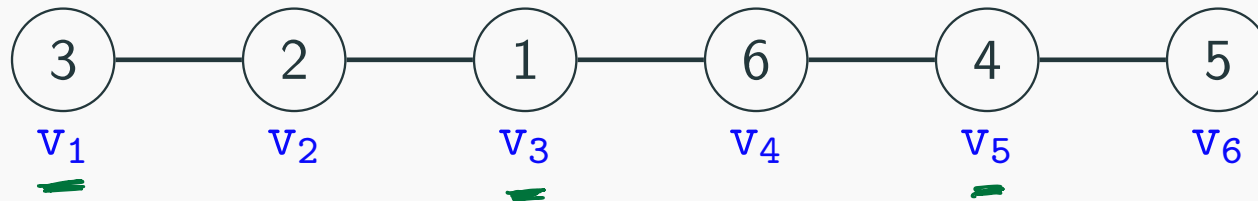
Input

set of vertices that are not connected by an edge

A path graph G with n vertices, $V = \{v_1, \dots, v_n\}$, with weights $w_1, \dots, w_n$.

Output

An independent set $S \subset V$ with max sum of weights $A = \sum_{v \in S} w_v$



Weighted Independent Set (WIS) on a path graph

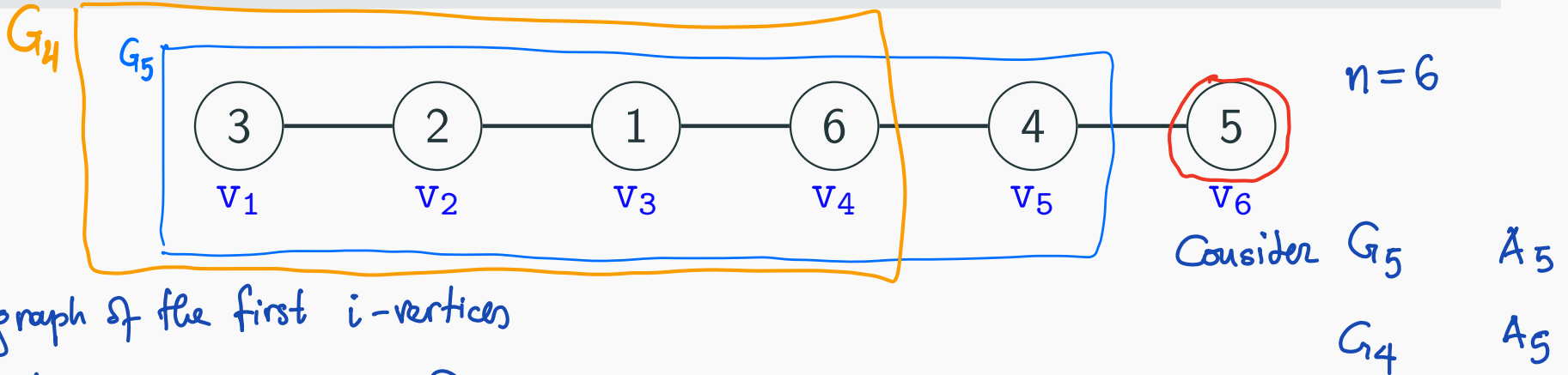
Input

set of vertices that are not connected by an edge

A path graph G with n vertices, $V = \{v_1, \dots, v_n\}$, with weights $w_1, \dots, w_n$.

Output

An independent set $S \subset V$ with max sum of weights $A = \sum_{v \in S} w_v$



G_i = graph of the first i -vertices

A_i = the Cost of WIS on G_i

goal is to compute A_n

Q: what do we do with v_6 ?

Weighted Independent Set (WIS) on a path graph

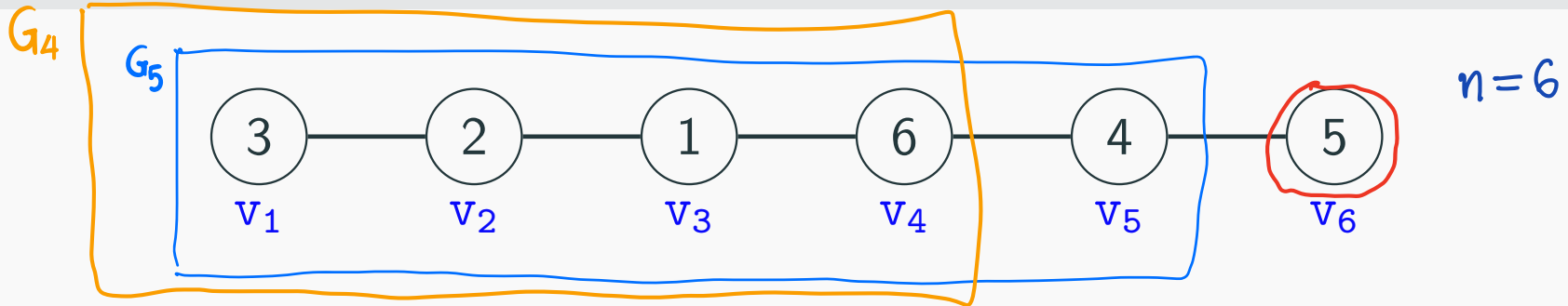
Input

set of vertices that are not connected by an edge

A path graph G with n vertices, $V = \{v_1, \dots, v_n\}$, with weights $w_1, \dots, w_n$.

Output

An independent set $S \subset V$ with max sum of weights $A = \sum_{v \in S} w_v$



G_i = graph of the first i -vertices

A_i = the Cost of WIS on G_i

goal is to compute A_n

Q: what do we do with v_6 ?

v_6 is included

$$A_6 = A_4 + w_6$$

v_6 is not included

$$A_6 = A_5$$

v_5 is included
otherwise the
cost is as in
 G_4 and A_4

$$A_i = \max \{ A_{i-1}, A_{i-2} + w_i \}$$

$$A_6 = \max \{ A_5, A_4 + w_6 \}$$

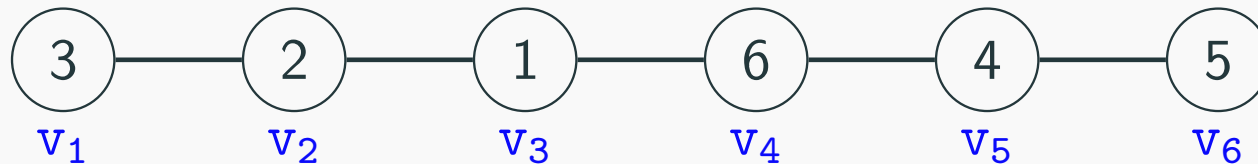
Weighted Independent Set (WIS) on a path graph

Input

A path graph G with n vertices, $V = \{v_1, \dots, v_n\}$, with weights $w_1, \dots, w_n$.

Output

An independent set $S \subset V$ with max sum of weights $A = \sum_{v \in S} w_v$



WIS Recurrence

For $i = 2, 3, \dots, n$

$$A_i = \max\{A_{i-1}, A_{i-2} + w_i\} \quad \text{with } A_0 = 0, A_1 = w_1$$

How do we fill the 1D array. Why?
How?
Is it 1D?

initial conditions
why they are two?

WIS: compute the cost

1D dynamic programming

$$A_0 = 0 \quad A_1 = w_1 = 3$$

$$A_2 = \max\{A_1, A_0 + w_2\} = \max\{3, 0 + 2\} = 3$$

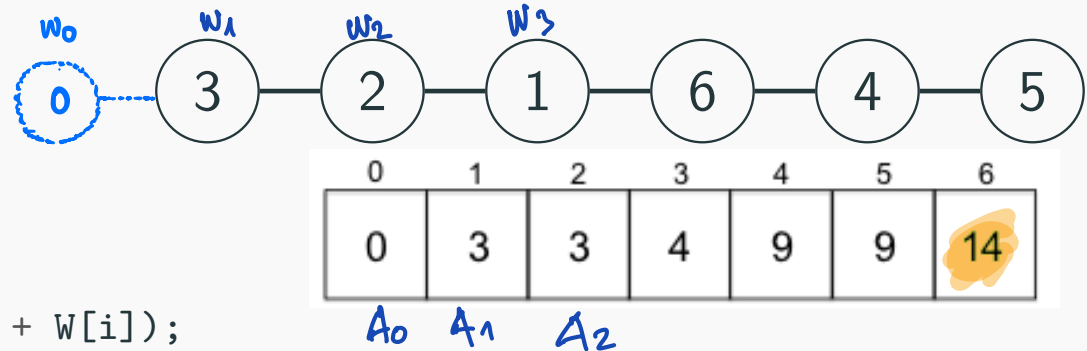
$$A_3 = \max\{A_2, A_1 + w_3\} = \max\{3, 3 + 1\} = 4$$

```
int n = 6;  
int W[n] = {3, 2, 1, 6, 4, 5}; // the array of the weights.  
int A[n];
```

```
A[0] = 0;  
A[1] = W[1];
```

```
for (int i = 2; i < n; ++i) {  
    A[i] = std::max(A[i-1], A[i-2] + W[i]);  
}
```

```
return A[n];
```



WIS Recurrence

For $i = 2, 3, \dots, n$

$$A_i = \max\{A_{i-1}, A_{i-2} + w_i\} \quad \text{with } A_0 = 0, A_1 = w_1$$

Complexity? $O(n)$ ops

WIS: Reconstruction *(Run the algorithm backwards)*

```
std::vector<int> S; // The set of vertices of the optimal solution
```

```
int i = n;
```

```
while (i >= 2) {
```

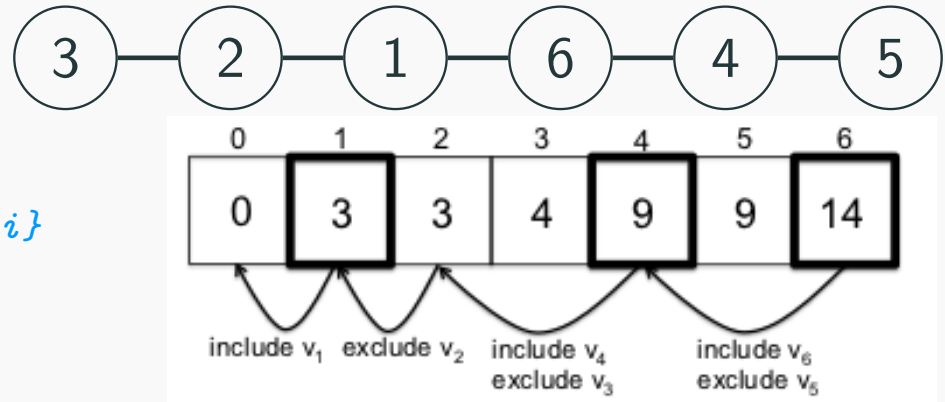
v_i is NOT included

```
    if (A[i-1] >= A[i-2] + W[i]) {  
        i = i - 1;  
    } else {  
        S.push_back(i) ; // S = S union {v_i}  
        i = i - 2;  
    }
```

```
    if (i = 1) {  
        S.push_back(1);  
    }
```

```
}
```

```
return S;
```



Complexity : $O(n)$ why?

WIS Recurrence

For $i = 2, 3, \dots, n$

$$A_i = \max\{A_{i-1}, A_{i-2} + w_i\} \quad \text{with } A_0 = 0, A_1 = w_1$$

Complexity of DP

General formula

$$\underbrace{S(n)}_{\text{\# subproblems}} \times \underbrace{T(n)}_{\text{time per subproblem}} + \underbrace{P(n)}_{\text{post-processing}}$$

Complexity of DP

General formula

$$\underbrace{S(n)}_{\text{\# subproblems}} \times \underbrace{T(n)}_{\text{time per subproblem}} + \underbrace{P(n)}_{\text{post-processing}}$$

$O(n)$, $O(1)$ + $O(n)$ = $O(n)$

```
A[0] = 0;
```

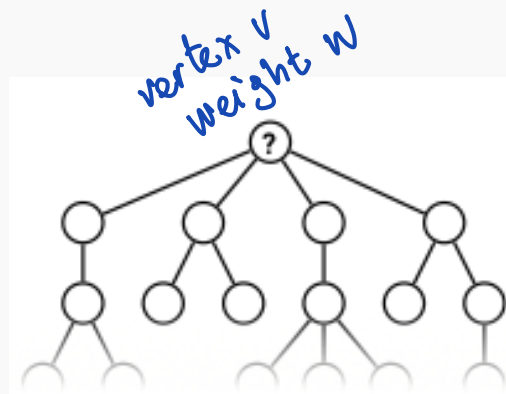
```
A[1] = W[1];
```

```
for (int i = 2; i < n; ++i) {  
    A[i] = std::max(A[i-1], A[i-1] + W[i]);  
}  
return A[n];
```

→ going backwards
to find the solution

WIS on a tree graph with n vertices and $n-1$ edges

Dynamic Programming on trees

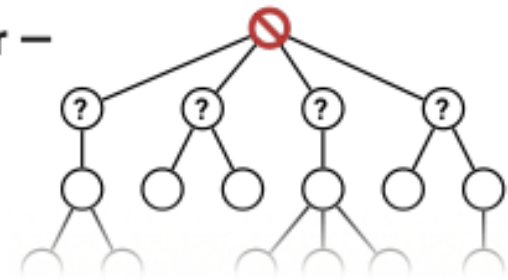


v is selected
exclude the children

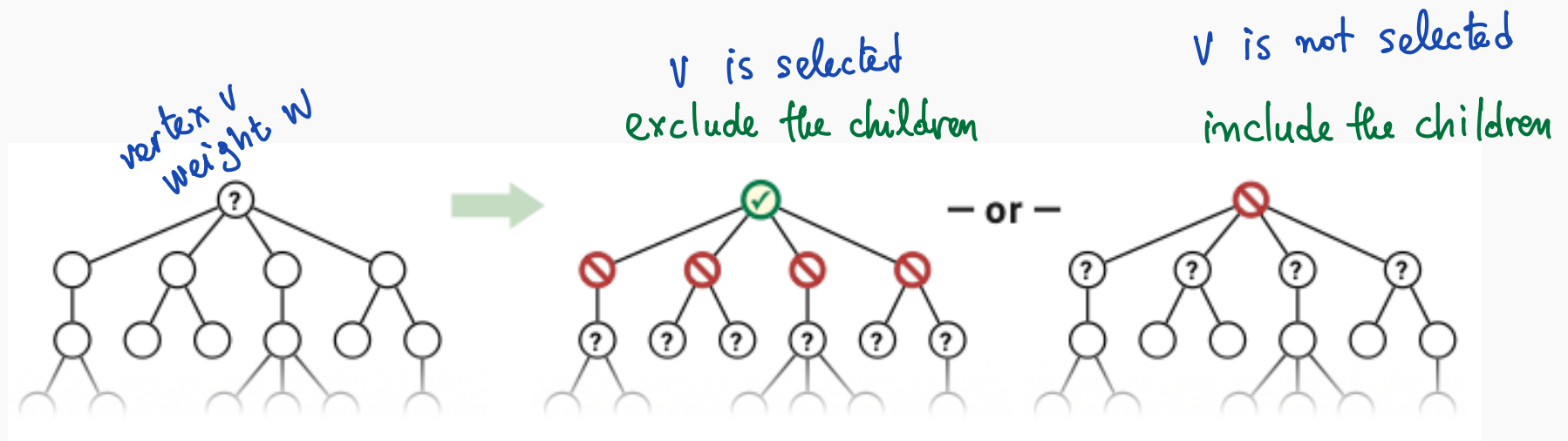


— or —

v is not selected
include the children

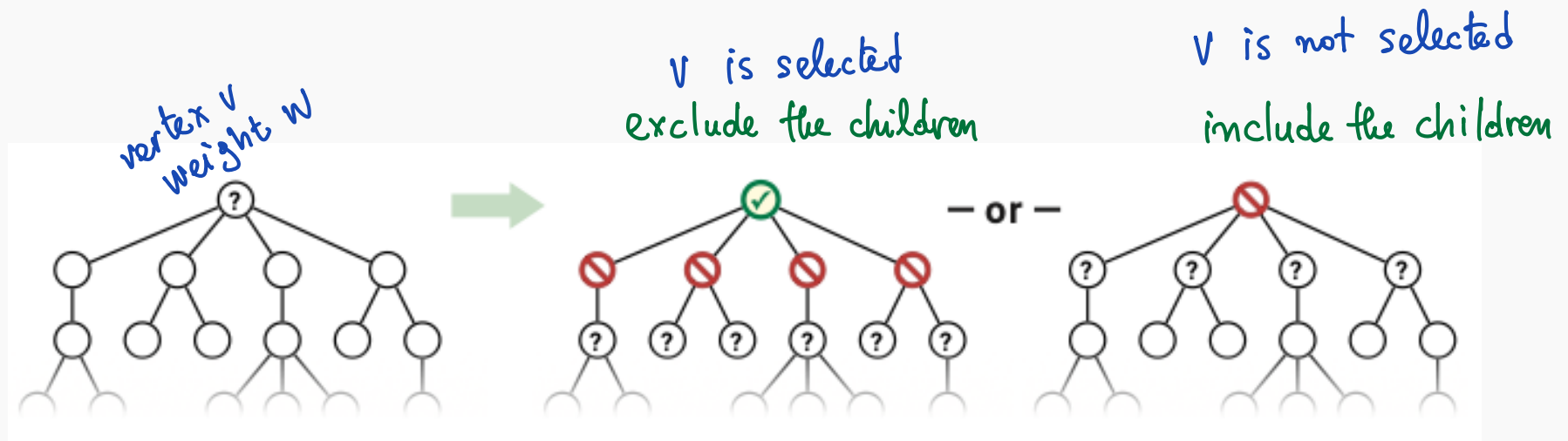


WIS on a tree graph with n vertices and $n-1$ edges



$$WIST(v) = \max \left\{ w + \sum_{\substack{u \text{ is a} \\ \text{grandchildren}}} WIST(u), \sum_{\substack{u \text{ is a} \\ \text{child}}} WIST(u) \right\}$$

WIS on a tree graph with n vertices and $n-1$ edges



$$WIST(v) = \max \left\{ w + \sum_{\substack{u \text{ is a} \\ \text{grandchildren}}} WIST(u), \sum_{\substack{u \text{ is a} \\ \text{child}}} WIST(u) \right\}$$

WIS

for general graphs : NP-hard

Complexity : $O(n)$ why?

An interesting problem $x_1, \dots, x_n$ find $a_i > 0$ s.t. $\sum_{i=1}^n a_i x_i < N$

TQ: The stamps problem

- Given a large collection of stamps of 1€, 4€, and 5€ and an amount N .
- Choose the fewer possible number of stamps with total cost N .

An interesting problem $x_1, \dots, x_n$ find $a_i > 0$ s.t. $\sum_{i=1}^n a_i x_i < N$

$M[i]$ = # stamps having total value i

for $0 \leq i \leq N$

$$M[i] = \min \begin{cases} 0 & i = 0 \\ 1 + M[i-5] & i \geq 5 \\ 1 + M[i-4] & i \geq 4 \\ 1 + M[i-1] & i \geq 1 \end{cases}$$

$\begin{matrix} \text{I put something with value 5} \\ 4 \\ 1 \end{matrix}$

$$M_0 = 0$$

$$M_1 = 1 + M_0 = 1$$

$$M_2 = 1 + M_1 = 1 + 1 = 2$$

$$M_3 = 1 + M_2 = 1 + 2 = 3$$

$$M_4 = \min \begin{cases} 1 + M_0 = 1 + 0 = 1 \\ 1 + M_3 = 1 + 3 = 4 \end{cases}$$

TQ: The stamps problem

- Given a large collection of stamps of 1€, 4€, and 5€ and an amount N .
- Choose the fewer possible number of stamps with total cost N .

An interesting problem $x_1, \dots, x_n$ find $a_i > 0$ s.t. $\sum_{i=1}^n a_i x_i < N$

$M[i]$ = # stamps having total value i

for $0 \leq i \leq N$

$$M[i] = \min \begin{cases} 0 & i = 0 \\ 1 + M[i-5] & i \geq 5 \text{ I put something with value 5} \\ 1 + M[i-4] & i \geq 4 \text{ 4} \\ 1 + M[i-1] & i \geq 1 \text{ 1} \end{cases}$$

TQ: The stamps problem

- Given a large collection of stamps of 1€, 4€, and 5€ and an amount N .
- Choose the fewer possible number of stamps with total cost N .

what if there are n values
 $v_1, \dots, v_n$

$$M[i] = 1 + \min_{1 \leq j \leq n} \{ M[\underbrace{i - v_j}_{\text{if } \geq 0}] \}$$

Contents

Fibonacci

WIS

LIS

Edit distance

Knapsack



Longest Increasing Subsequence 3 4 1 2 5 | 7 6 8 9

LIS

Given a sequence $S = (S_1, S_2, \dots, S_n) \in \mathbb{N}$ find the maximum number, k , of indices $1 \leq i_1 < i_2 < \dots < i_k \leq n$, such that $S_{i_1} \leq S_{i_2} \leq \dots \leq S_{i_k}$.

Def: $\text{LIS}(i, j)$ = length of the longest increasing subsequence of $S[j, \dots, n]$ in which every element is larger than $S[i]$.

Longest Increasing Subsequence

3 4 1 2 5 | 7 6 8 9

LIS

Given a sequence $S = (S_1, S_2, \dots, S_n) \in \mathbb{N}$ find the maximum number, k , of indices $1 \leq i_1 < i_2 < \dots < i_k \leq n$, such that $S_{i_1} \leq S_{i_2} \leq \dots \leq S_{i_k}$.

Def: $LIS(i, j) =$ length of the longest increasing subsequence of $S[j, \dots, n]$ in which every element is larger than $S[i]$.

$$LIS(i, j) = \begin{cases} \cdot \text{if } j > n, \text{ then } 0 \\ \cdot \text{if } S_i \geq S_j \text{ then } LIS(i, j+1) \\ \quad S_j \text{ does not contribute to the solution} \\ \cdot \text{if } S_i < S_j \text{ then} \\ \quad \max \left\{ \underbrace{LIS(j, j+1) + 1}_{S_j \text{ contributes}}, \underbrace{LIS(i, j+1)}_{S_j \text{ does not contribute}} \right\} \end{cases}$$



$LIS(i, j)$ is the length of a subset in $S[j, n]$ that has element greater than S_i

Complexity: $O(n^2)$

How do I fill the matrix?

Longest Increasing Subsequence

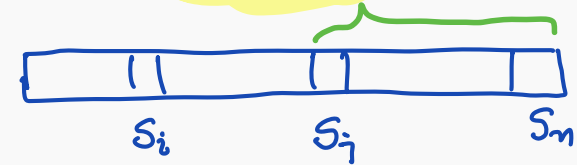
3 4 1 2 5 | 7 6 8 9

LIS

Given a sequence $S = (S_1, S_2, \dots, S_n) \in \mathbb{N}$ find the maximum number, k , of indices $1 \leq i_1 < i_2 < \dots < i_k \leq n$, such that $S_{i_1} \leq S_{i_2} \leq \dots \leq S_{i_k}$.

Def: $LIS(i, j) =$ length of the longest increasing subsequence of $S[j, \dots, n]$ in which every element is larger than $S[i]$.

$$LIS(i, j) = \begin{cases} \cdot \text{if } j > n, \text{ then } 0 \\ \cdot \text{if } S_i \geq S_j \text{ then } LIS(i, j+1) \\ \quad S_j \text{ does not contribute to the solution} \\ \cdot \text{if } S_i < S_j \text{ then} \\ \quad \max \left\{ \underbrace{LIS(j, j+1) + 1}_{S_j \text{ contributes}}, \underbrace{LIS(i, j+1)}_{S_j \text{ does not contribute}} \right\} \end{cases}$$



$LIS(i, j)$ is the length of a subset in $S[j..n]$ that has element greater than S_i

Complexity: $O(n^2)$

How do I fill the matrix?

for $j \leftarrow n$ downto 1
for $i \leftarrow 0$ to $j-1$ why?

Some common problems

TQ: Longest Common Subsequence

X_i = subseq up to i
 x_i = better at pos = i

$X =$

h	e	l	l	o	w	o	r	l	d
---	---	---	---	---	---	---	---	---	---

 m

Y_j and y_j

$Y =$

h	e	y	w	o	r	l	d
---	---	---	---	---	---	---	---

 length n

TQ: Longest Palindromic Subsequence

$A =$

a	a	b	c	d	e	b	a	z
---	---	---	---	---	---	---	---	---

Some common problems

$$LCS(X_i, Y_j) = \begin{cases} \emptyset & \text{if } i=0 \text{ or } j=0 \\ LCS(X_{i-1}, Y_{j-1}) + x_i & \text{if } x_i = y_j \\ \max\{LCS(X_i, Y_{j-1}), LCS(X_{i-1}, Y_j)\} & \text{if } x_i \neq y_j \end{cases}$$

TQ: Longest Common Subsequence

X_i = subseq up to i
 x_i = letter at pos = i

$X =$

h	e	l	l	o	w	o	r	l	d
---	---	---	---	---	---	---	---	---	---

 m

Y_j and y_j

$Y =$

h	e	y	w	o	r	l	d
---	---	---	---	---	---	---	---

 length n

Complexity $O(mn)$

How do we fill the matrix?

TQ: Longest Palindromic Subsequence

$A =$

a	a	b	c	d	e	b	a	z
---	---	---	---	---	---	---	---	---

Contents

Fibonacci

WIS

LIS

Edit distance

Knapsack



String similarity

minimal #operations
to transform one string
to another

← which ones? { removal
insertion
(substitution)

How similar are two strings?

- occurrence
- occurrence

Insert gaps

o	c	u	r	r	a	n	c	e	-
o	c	c	u	r	r	e	n	c	e

6 mismatches, 1 gap

o	c	-	u	r	r	a	n	c	e
o	c	c	u	r	r	e	n	c	e

1 mismatch, 1 gap

o	c	-	u	r	r	-	a	n	c	e
o	c	c	u	r	r	e	-	n	c	e

0 mismatches, 3 gaps

Sequence alignment :: edit distance

Applications:

- Basis of Unix *diff*.
- Speech recognition.
- Computational biology. *sequences of DNA & RNA*

Edit distance [Levenshtein 1966, Needleman-Wunsch 1970]

- penalty for gap δ ; penalty for mismatch α_{pq}
 - Cost = sum of gaps and mismatch penalties.
- $\alpha_{pq} = \begin{cases} 1 & \text{if } p \neq q \\ 0 & \text{if } p = q \end{cases}$

C T G A C C T A C C T

C C T G A C T A C A T

$$\alpha_{TC} + \alpha_{GT} + \alpha_{AG} + 2\alpha_{CA}$$

- C T G A C C T A C C T

C C T G A C - T A C A T

$$2\delta + \alpha_{CA}$$

Sequence alignment

Given two strings $X = x_1, x_2, \dots, x_m$ and $Y = y_1, y_2, \dots, y_n$, find an **alignment** of minimum cost.

$X_i = x_1, x_2, \dots, x_i$ and $Y_j = y_1, y_2, \dots, y_j$

Sequence alignment

Given two strings $X = x_1, x_2, \dots, x_m$ and $Y = y_1, y_2, \dots, y_n$, find an **alignment** of minimum cost.

$X_i = x_1, x_2, \dots, x_i$ and $Y_j = y_1, y_2, \dots, y_j$

① Consider the alignment of X_i and Y_j

$\left. \begin{array}{l} X_i + \text{gaps} \\ Y_j + \text{gaps} \end{array} \right\} \text{ has cost } C_{ij}$

Sequence alignment

Given two strings $X = x_1, x_2, \dots, x_m$ and $Y = y_1, y_2, \dots, y_n$, find an **alignment** of minimum cost.

① Consider the alignment of X_i and Y_j

$X_i = x_1, x_2, \dots, x_i$ and $Y_j = y_1, y_2, \dots, y_j$

$\begin{array}{|l} X_i + \text{gaps} \\ Y_j + \text{gaps} \end{array} \left. \vphantom{\begin{array}{|l} X_i + \text{gaps} \\ Y_j + \text{gaps} \end{array}} \right\} \text{ has cost } C_{ij}$

② Q: How is the alignment of X_i, Y_j at the last position?

$\begin{array}{ccc} x & x & \sqcup \\ y & \sqcup & y \end{array}$

Sequence alignment

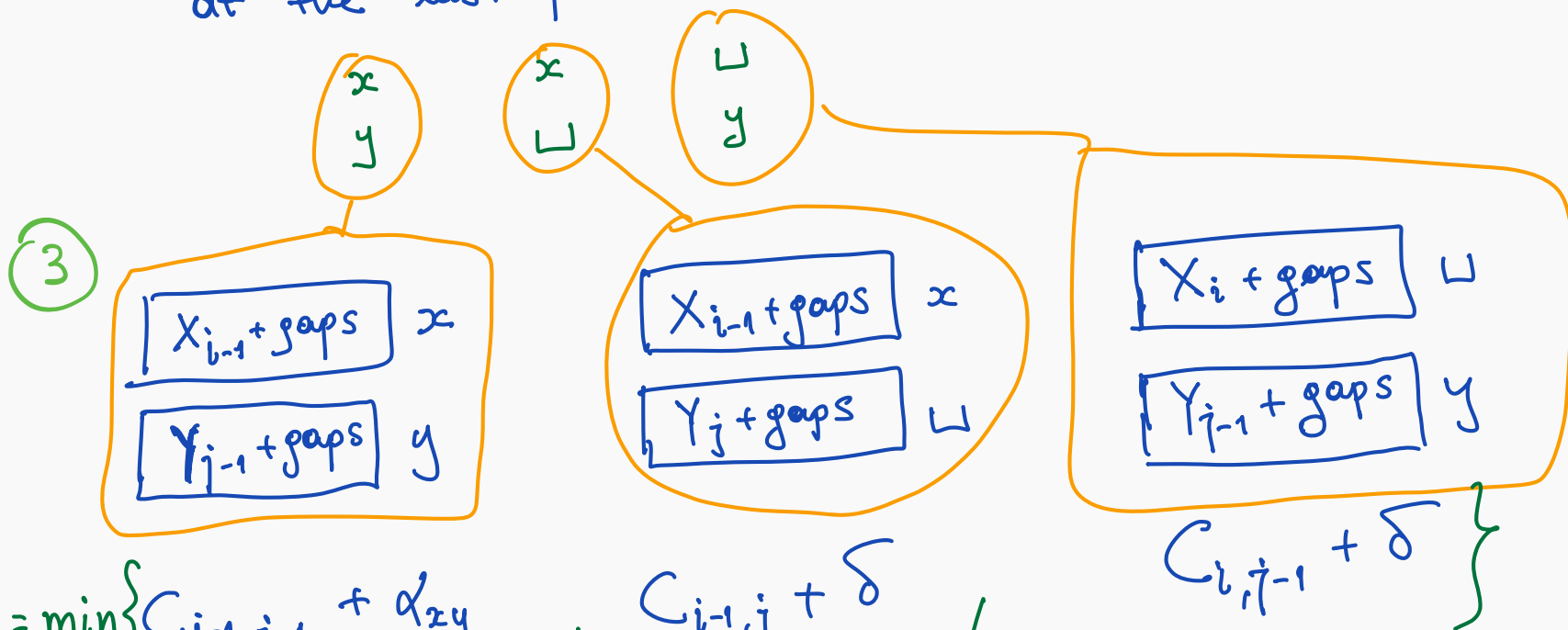
Given two strings $X = x_1, x_2, \dots, x_m$ and $Y = y_1, y_2, \dots, y_n$, find an **alignment** of minimum cost.

① Consider the alignment of X_i and Y_j

$X_i = x_1, x_2, \dots, x_i$ and $Y_j = y_1, y_2, \dots, y_j$

$\begin{array}{|c|} \hline X_i + \text{gaps} \\ \hline Y_j + \text{gaps} \\ \hline \end{array}$
 } has cost C_{ij}

② Q: How is the alignment of X_i, Y_j at the last position?



Complexity: If $m=n$ then $O(n^2)$ Can we do better? No unless SETH

space requirement: $O(n^2)$ this we can improve to $O(n \lg n)$

we can solve
k-SAT subexponentially

Sequence alignment

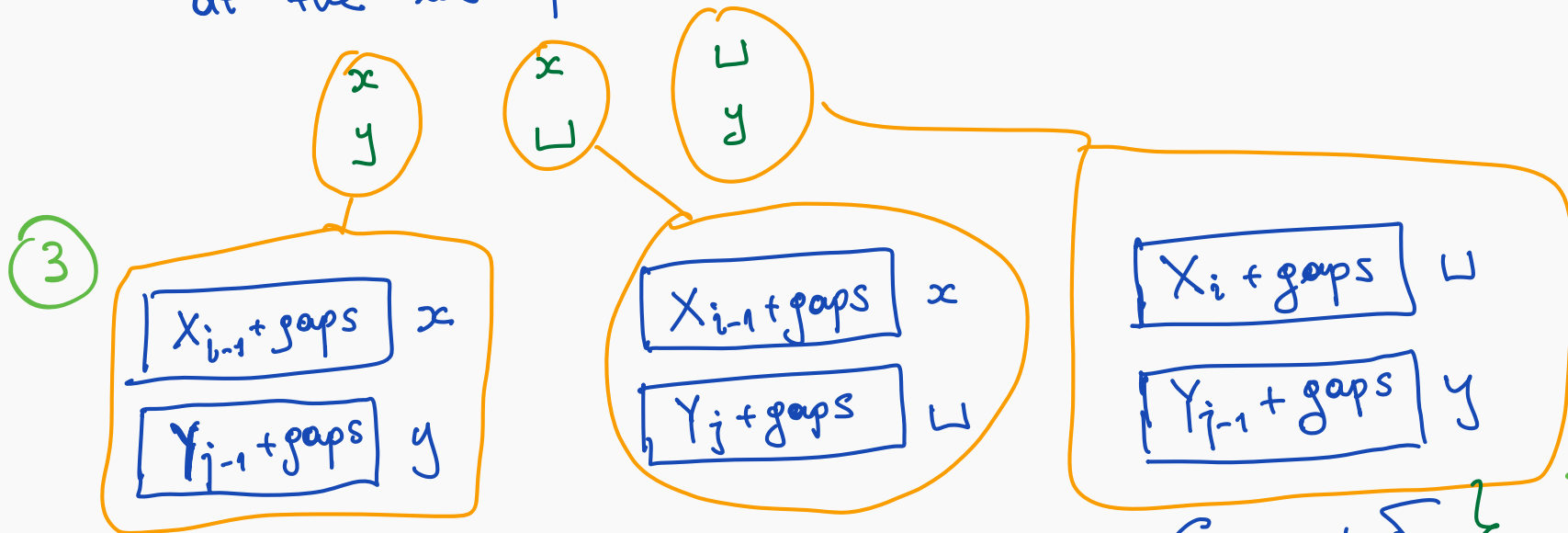
Given two strings $X = x_1, x_2, \dots, x_m$ and $Y = y_1, y_2, \dots, y_n$,
find an **alignment** of minimum cost.

① Consider the alignment of X_i and Y_j

$X_i = x_1, x_2, \dots, x_i$ and $Y_j = y_1, y_2, \dots, y_j$

$\begin{array}{|c|} \hline X_i + \text{gaps} \\ \hline Y_j + \text{gaps} \\ \hline \end{array}$ } has cost C_{ij}

② Q: How is the alignment of X_i, Y_j
at the last position?



$$C_{ij} = \min \left\{ C_{i-1, j-1} + d_{xy}, \quad C_{i-1, j} + \delta, \quad C_{i, j-1} + \delta \right\}$$

↓ subs → del ↓ ins

for $i=1$ to m
for $j=1$ to n

$O(mn)$

Contents

Fibonacci

WIS

LIS

Edit distance

Knapsack

The Knapsack problem

Input

Consider n items with values $v_1, \dots, v_n$ and sizes $s_1, \dots, s_n$.

A knapsack capacity C .

Output

A subset $S \subseteq \{1, 2, \dots, n\}$ that

maximizes $\sum_{i \in S} v_i$ and the total size is $\sum_{i \in S} s_i \leq C$.

Example

$$C = 6$$

Item	Value	Size
1	3	4
2	2	3
3	4	2
4	4	3

$$\text{size } 2+3 = 5 < 6 = C$$

$$\text{value } 4+4 = 8$$

Making a false start

$\text{OPT}(i)$ = max profit subset of items $1, \dots, i$.

- Case 1: OPT does not select item i .
 - OPT selects best of $\{1, 2, \dots, i - 1\}$
- Case 2: OPT selects item i .
 - selecting item i does not immediately imply that we will have to reject other items
 - without knowing what other items were selected before i , we do not even know if we have enough room for i

Thus, we need more subproblems!

Knapsack recurrence

the important
additional param
↓

$\text{OPT}(i)$ = max profit subset of items $1, \dots, i$ with capacity limit c .

- Case 1: OPT does not select item i .
 - OPT selects best of $\{1, 2, \dots, i-1\}$ using capacity limit c .
- Case 2: OPT selects item i .
 - new capacity limit $c - s_i$.
 - OPT selects best of $\{1, 2, \dots, i-1\}$ using the new capacity limit $c - s_i$.

update the capacity
at each step

Recurrence

for $0 \leq c \leq C$

$$\text{OPT}(i, c) = \begin{cases} 0 & \text{if } i = 0 \\ \text{OPT}(i-1, c) & \text{if } s_i > c \\ \max\{\text{OPT}(i-1, c), v_i + \text{OPT}(i-1, c - s_i)\} & \text{otherwise} \end{cases}$$

i is NOT
included

i is included

for $0 \leq c \leq C$ do
for $0 \leq i \leq n$ do
 :
 :

Fill the array of Knapsack

```
for (int c = 2; c <= C; ++c) {  
    A[0, c] = 0;  
}  
  
for (int i = 1; i < n; ++i) {  
    for (int c = 1; c <= C; ++c) {  
        if (s[i] > c) {  
            A[i, c] = A[i-1, c];  
        } else {  
            A[i, w] = std::max( A[i-1, c], A[i-1, c-s[i]] + V[i]);  
        }  
    }  
}  
return A[n, C];
```

INPUT: n and $\lg C$

TQ

Complexity: $\mathcal{O}(\underline{n}\underline{C})$. Is it polynomial? *Pseudo-polynomial*

TQ

When n do we use dynamic programming instead of a greedy algorithm?

- Compute the OPT using prev subproblems
- Find the actual OPT

- the OPT at the current state
- Hope to find the OPT
- Local OPT leads to global OPT

NEVER without a proof

Unbounded Knapsack problem

Input

Consider n items with values $v_1, \dots, v_n$ and sizes $s_1, \dots, s_n$.

A knapsack capacity C . *We allow repetitions*

1-dim array

Output

Non-negative numbers $\{q_1, \dots, q_n\}$ that
maximize $\sum_{i \in S} q_i v_i$ and the total size is $\sum_{i \in S} q_i s_i \leq C$.

Unbounded Knapsack problem

Input

Consider n items with values $v_1, \dots, v_n$ and sizes $s_1, \dots, s_n$.

A knapsack capacity C . *We allow repetitions*

Output

1-dim array

Non-negative numbers $\{q_1, \dots, q_n\}$ that maximize $\sum_{i \in S} q_i v_i$ and the total size is $\sum_{i \in S} q_i s_i \leq C$.

*for $0 \leq i \leq W$
 for $0 \leq j \leq N$
 if $wt[j] < i$ then
 $K[i] = \max \{ K[i] + K[i - wt[j]] + val[j] \}$*

$O((W+1)N)$ time

$O(W+1)$ space

similar to stamps problem

The Dynamic Programming paradigm

- Formulate the problem recursively
Specification and Solution (in English or in French first!)
- Build the solutions of the recurrence from bottom up
 - Identify the subproblems.
 - Choose a **memoization** data structure (usually a 1- or 2-dim array).
 - Identify dependencies.
 - Find a good evaluation order.
 - Analyze space and time complexity.
 - Write down the algorithm.

DP and D&C

Differences between DP and D&C

- DP can (sub)divide in multiple ways.
- DP can benefit from memoization.
- DP can result an exponential improvement in complexity bounds.
- in DP proving correctness is not straightforward (exchange argument)
- DP can deal with subproblems of different size.
- DP is a generalization of D&C !